

Fast Approximations of Betweenness Centrality with Graph Neural Networks

Sunil Kumar Maurya
skmaurya@net.c.titech.ac.jp
Tokyo Institute of Technology
Tokyo, Japan

Xin Liu
xin.liu@aist.go.jp
AIRC, AIST
Tokyo, Japan

Tsuyoshi Murata
murata@c.titech.ac.jp
Tokyo Institute of Technology
Tokyo, Japan

ABSTRACT

Betweenness centrality is an important measure to find out influential nodes in networks in terms of information spread and connectivity. However, the exact calculation of betweenness centrality is computationally expensive. Although researchers have proposed approximation methods, they are either less efficient, or suboptimal, or both. In this paper, we present a Graph Neural Network(GNN) based inductive framework which uses constrained message passing of node features to approximate betweenness centrality. As far as we know, we are the first to propose a GNN based model to accomplish this task. We demonstrate that our approach dramatically outperforms current techniques while taking less amount of time through extensive experiments on a series of real-world datasets. Source code at <https://github.com/sunilkmaurya/GNN-Bet>

ACM Reference format:

Sunil Kumar Maurya, Xin Liu, and Tsuyoshi Murata. 2019. Fast Approximations of Betweenness Centrality with Graph Neural Networks. In *Proceedings of The 28th ACM International Conference on Information and Knowledge Management, Beijing, China, November 3–7, 2019 (CIKM '19)*, 4 pages. <https://doi.org/10.1145/3357384.3358080>

1 INTRODUCTION

Node ranking is one of the important measures to find out influential nodes in graphs in terms of information spread and connectivity. One of the ways a node can be ranked is based on its ability to control the spread of information between other nodes[9]. Ranking based on this criterion is known as betweenness centrality[4].

The main idea behind betweenness centrality is that a given node is central if many shortest paths pass through it. A high value of betweenness centrality for a node means that this node lies on many shortest paths between other nodes and helps pass information between other nodes. In other words, these nodes act as "bridges" in the graph. Removing these nodes from the graph will lead to longer paths for information to travel between other nodes. For a given graph $G = (V, E)$, betweenness centrality of a node v in a given network is defined as

$$BC(v) = \sum_{u \neq v \neq w} \frac{\sigma_{uw}(v)}{\sigma_{uw}} \quad (1)$$

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions.acm.org.

CIKM '19, November 3–7, 2019, Beijing, China
© 2019 Association for Computing Machinery.
ACM ISBN 978-1-4503-6976-3/19/11...\$15.00
<https://doi.org/10.1145/3357384.3358080>

where σ_{uw} is the total number of shortest paths from node u to w and $\sigma_{uw}(v)$ is the number of paths from u to w going through v .

Brandes[2] gives an exact algorithm to calculate betweenness by finding shortest paths starting from each node $v \in V$ using BFS and aggregating the contributions of nodes on these paths for all nodes. Calculating exact betweenness centrality value for nodes is computationally expensive and has time complexity of $\Theta(|V||E|)$. Hence it is prohibitive for large graphs.

In most of the cases, we do not require exact calculation of rankings of nodes, so an approximation is sufficient for tasks in hand. Many algorithms have been proposed to approximate the calculations of betweenness.

Geisberger[5] proposes a method to approximate betweenness centrality by sampling k starting nodes(pivots) to find the shortest paths and estimate betweenness centrality for all nodes. Riondato[10] takes a different approach while providing a theoretical guarantee. r shortest paths are chosen between randomly sampled source-target node pairs and betweenness is approximated based on these paths. Borassi's algorithm [1] provides an adaptive sampling technique for sampling shortest paths which provides a faster computation of betweenness within a given absolute error.

However, these methods are either less efficient or suboptimal or both. In this work, we provide Graph Neural Network(GNN) based inductive framework to approximate the betweenness centrality ranking of nodes. We take the intuition of the exact betweenness algorithm of aggregating nodes contribution by performing BFS from each node. In GNN each node aggregates the features of nodes in multi-hop neighborhood. We use this feature aggregation to learn how many nodes are reachable to a specific node, although we put constraints on how the feature information flows in the graph. We propose a GNN based inductive model which can train on one set and test on another set of graphs with varying structures. The benefits of using a GNN is that we can use computational power of GPUs and once trained the time for model inference is really small. In our proposed approach, we focus on approximating the rankings of nodes rather than finding the absolute betweenness values.

2 MODEL CONSTRUCTION

In this section, we first discuss message passing in graph neural networks. Then we propose a framework to approximate betweenness ranking based on message passing mechanism.

2.1 Message Passing Neural Network

There are many variants of graph neural networks available in literature[8][7][3]. All these models can be described under the general Message Passing Neural Network(MPNN) framework[6] as their fundamental operation of node feature aggregation is similar.

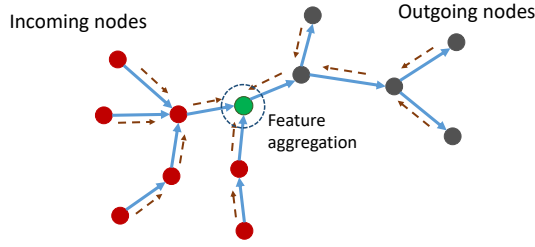


Figure 1: Each node aggregates the features of incoming nodes and outgoing nodes separately

Let $G = (V, E)$ denotes the graph with given node feature vectors as X_v for $v \in V$. At each layer, each node aggregates the features of its neighbors, which can also be considered as message passing phase. Node feature vector is updated by combining its own feature vectors with aggregated features from its neighbors. After K number of iterations, the final node representation vector captures structural information of K -hop neighbors as well as their features. Mathematically, GNNs can be formulated as,

$$a_v^{(k)} = \text{AGGREGATE}^{(k)}(\{h_u^{(k-1)} : u \in \mathcal{N}\}) \quad (2)$$

$$h_v^{(k)} = \text{COMBINE}^{(k)}(h_v^{(k-1)}, a_v^{(k-1)}) \quad (3)$$

In equation (2), feature vector of neighbors of nodes are aggregated. The AGGREGATE operation can vary based on the model requirement and can be summing, averaging or max-pooling of feature vectors. In equation (3), the aggregate feature vector is then added to the node feature vector. After each iteration, non-linearity like ReLU is applied. Final embedding is then used for training with respect to some objectives like node classification or link prediction.

2.2 Proposed Framework

From the construction of GNN as described above, we observe that each node accumulates the feature vectors of its multi-hop neighbors with increasing number of layers or conversely each node's feature is spread across network with consecutive layers. We use this feature information flow to model the paths in the graph. A node that is connected to many other nodes through paths of varying length is able to accumulate more features across the graph.

Our proposed model GNN-Bet is similar to other general GNNs but with some key differences which will be described below.

2.2.1 Adjacency Matrix. In betweenness centrality calculation, many nodes have zero betweenness centrality, as no shortest paths going through them. This condition arises when either a node only has a single neighbor and there is no path going through the node or node's neighbors form cliques such that no shortest paths go through the node. We identify such nodes beforehand and modify the adjacency matrix so that these nodes are not able to aggregate any features from neighbors. This can be done by simply setting the node's corresponding row to zero in the adjacency matrix. This ensures no feature information flow occurs through the edges connected to these nodes. Features of zero betweenness nodes are added to other nodes with non-zero betweenness only in case if they are

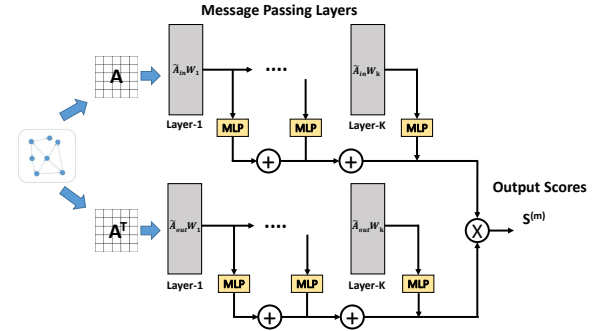


Figure 2: Proposed model diagram

terminal nodes in paths passing through non-zero betweenness nodes. We denote the new modified adjacency matrix as $\tilde{A}$.

2.2.2 Model Layer. Initial features of nodes are embedding lookup from the weight matrix. At the layer, node aggregates features of its k -hop neighbors. In our model, we define aggregation as sum of feature vectors. Unlike other GNN models, we do not add the node's own feature vectors with current layer neighborhood feature aggregation vector. Hence at each layer node has unique information about its neighborhood at k -hop. Then aggregated node features from each layer are mapped to a multilayer perceptron (MLP) unit to output a score vector for all nodes. We use a single MLP unit to output scores for all layers. In the end, we combine the scores obtained for each layer to get final scores for a node.

For a node in the given graph, there are two types of neighbors, in-degree neighbor and out-degree neighbors. We accumulate feature vectors of in-degree neighbors and out-degree neighbors for multiple hops. This aggregation approximates the information of incoming paths and outgoing paths to a given node. Hence using our proposed framework, we get two scores for all in-degree neighbors and out-degree neighbors. As the number of paths passing through a given node are the combinations of incoming paths and outgoing paths through the node, we multiply the in-degree and out-degree score in order to get the final score for each node.

2.2.3 Loss Function. We use a ranking loss function to get the loss for model predicted scores with respect to actual betweenness centrality scores. For model score, $S_i^{(m)}$ and actual betweenness score $S_i^{(b)}$, the loss function is defined as follows,

$$\text{Loss}(x, y) = \max(0, -y * (S_i^{(m)} - S_i^{(b)}) + \text{Margin}), \quad (4)$$

$$y = \begin{cases} 1 & \text{if } S_i^{(m)} \text{ should be ranked higher than } S_i^{(b)}. \\ -1 & \text{if } S_i^{(b)} \text{ should be ranked higher than } S_i^{(m)}. \end{cases}$$

3 EXPERIMENTS

In this section, we present the details of training and testing of the model. We evaluate our model on synthetic graphs as well as real-world graphs and compare its performance with other betweenness approximation techniques.

3.1 Training Setup

3.1.1 Hardware and Software Setup. All experiments were conducted on Intel Core i7-8700K CPU @ 3.70GHz machine with 12-cores, 64 GB RAM and NVIDIA GeForce GTX 1080Ti GPU with 12

Algorithm 1: GNN-Bet (Forward propagation)

Input : Directed Graph adjacency matrix A ; depth K ;
non-linearity ReLU

Output: Centrality value vector, $S^{(m)}$

```

1  $\tilde{A}_{\text{out-degree}} \leftarrow \text{ModifyAdjacency}(A)$ 
2  $\tilde{A}_{\text{in-degree}} \leftarrow \text{ModifyAdjacency}(A^T)$ 
3 for  $k = 1 \dots K$  do
4    $H_{\text{out-degree}}^{(k)} \leftarrow \text{ReLU}(\tilde{A}_{\text{out-degree}} W^{(k)})$ 
5    $H_{\text{in-degree}}^{(k)} \leftarrow \text{ReLU}(\tilde{A}_{\text{in-degree}} W^{(k)})$ 
6    $S_{\text{out-degree}}^{(k)} \leftarrow \text{MLP}(H_{\text{out-degree}}^{(k)})$ 
7    $S_{\text{in-degree}}^{(k)} \leftarrow \text{MLP}(H_{\text{in-degree}}^{(k)})$ 
8 end
9  $S_{\text{out-degree}} \leftarrow \sum_{k=1 \dots K} S_{\text{out-degree}}^{(k)}$ 
10  $S_{\text{in-degree}} \leftarrow \sum_{k=1 \dots K} S_{\text{in-degree}}^{(k)}$ 
11  $S^{(m)} \leftarrow S_{\text{out-degree}} \times S_{\text{in-degree}}$ 

```

GB graphics memory. Software frameworks used are PyTorch for implementing the proposed model, NetworkX and NetworkIt for graph generation and betweenness calculations.

3.1.2 Hyper-parameters. The model size is determined by the largest test graph size. For smaller graph inputs, adjacency matrix of input graph is placed along diagonal and rest is padded with zeros. We train the model with Adam as optimizer with learning parameter set to 0.005. We set the number of layers in our model to 4. Dimension of embedding is set as 12. In order to calculate loss value during training, we need to compare the node pair rankings. But the total combination of all node pairs can be really large and computationally prohibitive. In order to avoid this, we randomly sample node pairs equal to 20 times the number of nodes.

3.1.3 Training. We first divide given set of graphs into training dataset and test dataset. For graphs in both datasets, betweenness centrality of nodes are calculated. These values are used as target vectors to train the model and to evaluate the performance of the model during inference time for the test dataset. We remove the self-loops from the input graphs. But we keep the multi-edges, if present in the graphs. We train the model for 5-10 epochs and training time is roughly 10-15 minutes. Note that the test graphs are totally unseen during the training stage(inductive settings).

3.1.4 Evaluation Measure. In order to evaluate ranking quality of model output, we use Kendall's Tau rank correlation coefficient(refer as KT Score) as ranking measure. Value of coefficient is 1 when all pairs in two lists are ranked in the same order and -1 if they are in opposite order. In our case, we compare ranking of model output for nodes to the ranking of their real centrality values.

3.2 Model Evaluation

We evaluate and compare our proposed model to other betweenness approximation methods on synthetic as well as realworld graphs.

3.2.1 Dataset Preparation. For synthetics datasets, we generate three classes of directed graphs namely, Erdos-Renyi random

Table 1: Summary of real world datasets used

Dataset	#Nodes	#Edges
Wiki-Vote	7,115	103,689
P2p-Gnutella31	62,586	147,892
Soc-Epinions	75,879	508,837
Soc_Slashdot	77,350	516,575
Ego-Gplus	107,614	13,673,453
Email-EuAll	265,214	420,045
Web-Google	875,713	5,105,039
Wiki-Talk	1,140,149	7,833,140

graphs, Scale-free graphs and Gaussian random partition graphs. Sizes of graphs vary from 50,000 to 100,000 nodes. For each class, we generate 15 graphs with number of nodes and generation parameters set randomly so that we can have structural variations in graphs. For the training dataset, we take 5 graphs and create 500 training samples by randomly permutating the node sequence in adjacency matrices and their corresponding target betweenness vector. We train the model on the training set and evaluate the performance on 10 graphs in the test set.

For model evaluation on realworld datasets, we have taken 8 datasets from Stanford Large Network Dataset(SNAP) website. Table 1 shows the details for these datasets. Real world graphs have varying structural properties. Under the weak assumption that real world graphs have some scale-free graph properties, we trained our model on synthetic scale-free graphs. We created 5 synthetic directed scale-free graphs of 100,000 nodes each using NetworkX. From these 5 graphs, we generated 250 adjacency matrices by permutating the node sequences and corresponding betweenness centrality vectors. 200 of them were used for training and 50 were for validation. We performed the tests on real world dataset graphs.

3.2.2 Comparison Methods. We compare the performance of our model to other betweenness approximation methods. Algorithm proposed by Geisberger [5] is referred as GS, Riandato [10] as RK and Borassi [1] as BN. We also show the time taken by the exact algorithm provided by Brandes [2] as a reference to compare all approximate algorithms. For comparison techniques, we have set two parameters for RK and BN $\delta = 0.1$ and $\lambda = 0.005$ as used in the respective literature. We found that RK algorithm was taking too much time for approximation calculations of $\lambda < 0.02$, sometimes approaching/exceeding the exact betweenness centrality calculation time. So for large graphs we show results for RK for $\lambda = 0.02$ marked with '*'. For GS we need to provide the number of random nodes to be sampled. We set it to 8,192 which is the maximum number of samples used in the literature even for large graphs.

3.3 Results

Table 2 shows KT ranking of output score vector with respect to actual betweenness centrality values for all nodes. We also show time taken to calculate approximation by all methods and compare them with exact betweenness calculations. Time taken for our model is sum of time taken to identify nodes with zero betweenness and model inference time. For other algorithms, the execution time is shown. For synthetic graphs, average KT score and the average time taken for 10 test graphs are calculated and presented.

Table 2: KT ranking scores comparison with various betweenness approximation methods on real world and synthetic datasets

Dataset	KT Score				Time (s)				
	GS	RK	BN	GNN-Bet	Exact	GS	RK	BN	GNN-Bet
Wiki-Vote	0.866	0.844	0.900	0.967	0.8	7.1	1.6	0.2	0.1
P2p-Gnutella31	0.849	0.963	0.876	0.924	57.2	18.06	163.4	5.4	0.2
Soc-Epinions	0.723	0.826	0.687	0.891	222.1	86.5	567.9	6.8	0.5
Soc_Slashdot	0.766	0.793	0.639	0.909	206.7	71.2	510.3	5.96	0.9
Ego-Gplus	0.817	0.673	0.515	0.821	3845.7	979.4	7787.5	11.8	7.2
Email-EuAll	0.889	0.508	0.380	0.990	1091.5	99.8	985.1	8.1	1.1
Web-Google	0.755	0.366*	0.498	0.779	52422	2264	8548	314.4	8.5
Wiki-Talk	0.679	0.204*	0.271	0.979	32385	624.2	1115.4	32.4	3.6
Synthetic-ER	0.845±0.05	0.829±0.07	0.629±0.14	0.902±0.03	811.6±560.1	206.3±142.0	1354.2±765.7	20.2±3.3	0.6±0.3
Synthetic-SF	0.464±0.02	0.574±0.03	0.411±0.03	0.976±0.01	33.81±4.4	5.2±1.7	114.0±30.7	1.8±0.3	0.3±0.2
Synthetic-GP	0.562±0.06	0.342±0.17	0.141±0.08	0.899±0.04	27.8±12.5	2.9±1.4	98.0±30.7	1.5±0.4	0.6±0.2

We notice that RK takes a large amount of time to get good accuracy, BN is really fast but the ranking of nodes are suboptimal compared to other methods while GS performs relatively better with higher ranking accuracy compared to both. For both synthetic and realworld datasets, high KT scores show that our model can calculate good betweenness approximation ranking and outperforms all other methods while taking less amount of time.

3.4 Time comparison

As we can see from the results, BN is the fastest betweenness approximation method. In order to compare the speed of approximation of our proposed model with BN, we test them on 10 synthetic scale-free graphs with the number of nodes ranging from 100 thousand to one million. We set the size of model to accommodate for all input graphs, hence the time taken is proportional to the number of edges. Figure 3 shows the time taken by both methods. Our model is quite faster and scales well as compared to BN.

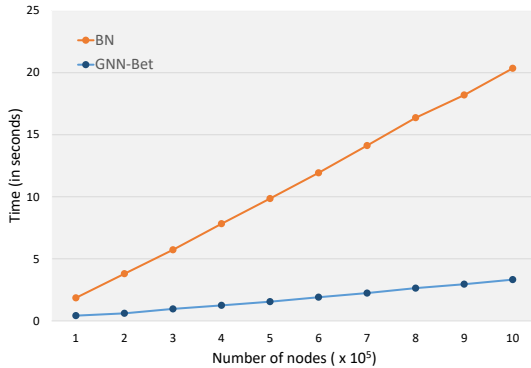


Figure 3: Time taken by model as number of nodes increases for synthetic scalefree graphs

3.5 Discussion

We have set number of layers in our model to four for all graph datasets to compare results on a single model. But number of layers can be chosen based on structure and diameter of the graph. Even though the model is trained on synthetic scale-free graphs, tests on realworld graph datasets show good approximation accuracy. This shows that the proposed model is inductive in nature. It is able to learn on one set of graphs and can predict node ranking scores on others. Also, the model is invariant to the node sequence

in adjacency matrix so we do not have any constraint of ordering the nodes to any specific sequence in graphs.

It is presumable that training and testing on graphs with similar structural properties is expected to give better results. As the model takes advantage of high performance GPU, the inference time is really small. Hence it can also be used in an online setting where the graph data is dynamic and there is a need to recalculate betweenness often.

4 CONCLUSION

In this paper, we propose a novel approach to solve the betweenness approximation problem in graphs using GNN. Using the constrained message passing scheme, we train graph neural network model to output ranking scores for nodes in correlation with betweenness centrality values. We compare the performance of the model with other state of the art betweenness approximation techniques and the experimental results show better performance of our model while taking significantly less time.

ACKNOWLEDGEMENT

This work was supported by JST CREST (Grant Number JPMJCR1687), JSPS Grant-in-Aid for Scientific Research(B) (Grant Number 17H01785), JSPS Grant-in-Aid for Early-Career Scientists(Grant Number 19K20352), and the New Energy and Industrial Technology Development Organization (NEDO).

REFERENCES

- [1] M. Borassi et al. KADABRA is an Adaptive Algorithm for Betweenness via Random Approximation. *J. Exp. Algorithmics*, 24(1), Feb. 2019.
- [2] U. Brandes. A faster algorithm for betweenness centrality. *The Journal of Mathematical Sociology*, 25(2), June 2001.
- [3] D. K. Duvenaud et al. Convolutional Networks on Graphs for Learning Molecular Fingerprints. In *NIPS*. 2015.
- [4] L. C. Freeman. A Set of Measures of Centrality Based on Betweenness. *Sociometry*, 40(1), 1977.
- [5] R. Geisberger et al. Better Approximation of Betweenness Centrality. In *Proceedings of the Meeting on Algorithm Engineering & Experiments*, 2008.
- [6] J. Gilmer et al. Neural Message Passing for Quantum Chemistry. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70, ICML'17*, 2017.
- [7] W. Hamilton et al. Inductive Representation Learning on Large Graphs. In I. Guyon et al., editors, *NIPS*. 2017.
- [8] T. N. Kipf et al. Semi-Supervised Classification with Graph Convolutional Networks. *ICLR*, 2017. arXiv: 1609.02907.
- [9] M. E. J. Newman. A measure of betweenness centrality based on random walks. *Social Networks*, 27(1), Jan. 2005.
- [10] M. Riondato et al. Fast approximation of betweenness centrality through sampling. *en. Data Mining and Knowledge Discovery*, 30(2), Mar. 2016.